Lecture 6: Inductive Types, Recursion, and Induction

Stefano M. Nicoletti

1 Lecture 6: Inductive Types, Recursion, and Induction

1.1 Aim of the lecture

In this lecture we define types with an explicit structure, recursive functions on their values, and proofs by induction. We also use `rfl`, `unfold`, `rw`, `rewrite`, and `simp`.

1.2 Inductive types

In earlier lectures, `Thing : Type` left the structure of the domain open. An inductive type instead lists the ways in which we can build its values. These ways are called *constructors*.

```
inductive TrafficLight where
  | red
  | yellow
  | green
```

The three constructors describe every possible value of `TrafficLight`. We can define a predicate by pattern matching:

```
def MustStop : TrafficLight → Prop
  | .red => True
  | .yellow => True
  | .green => False
```

Pattern matching gives a result for each constructor. To reason about every traffic light, `cases light` opens the cases `red`, `yellow`, and `green`.

```
theorem lecture06_traffic_light_has_a_colour (light : TrafficLight) :
    light = .red ∨ light = .yellow ∨ light = .green := by
  cases light with
  | red =>
      apply Or.inl
      rfl
  | yellow =>
      apply Or.inr
      apply Or.inl
      rfl
  | green =>
```

```
      apply Or.inr
      apply Or.inr
      rfl
```

1.3 Our natural numbers

We define a type of natural numbers without using `Nat`:

```
inductive Natural where
  | zero
  | succ (n : Natural)
```

```
open Natural

def one : Natural := succ zero
```

```
open Natural

def two : Natural := succ one
```

```
open Natural

def three : Natural := succ two
```

`zero` is a natural number. If `n` is a natural number, `succ n` is a natural number. Every finite value of `Natural` is a chain of `succ` ending in `zero`.

1.4 Recursion

A recursive definition solves a problem by using the result of the same problem on simpler data. For `Natural`, we distinguish the case `zero` from the case `succ n`.

```
open Natural

def add (m n : Natural) : Natural :=
  match n with
  | zero => m
  | succ n => succ (add m n)
```

```
open Natural

def double (n : Natural) : Natural :=
  match n with
  | zero => zero
  | succ n => succ (succ (double n))
```

`match n with` says that `add` analyses its second argument. In the `succ n` case, the recursive call uses the smaller natural number `n`. Lean accepts the definition because every recursive call moves towards a base case.

1.5 Computation, ‘rfl‘, and ‘unfold‘

Two expressions are definitionally equal when Lean can reduce them to the same expression. In these cases, `rfl` closes the goal:

```
open Natural

example : add one one = two := by
  rfl
```

```
open Natural

theorem lecture06_add_zero_right (m : Natural) :
    add m zero = m := by
  rfl
```

```
open Natural

theorem lecture06_add_succ_right (m n : Natural) :
    add m (succ n) = succ (add m n) := by
  rfl
```

```
open Natural

theorem lecture06_double_succ (n : Natural) :
    double (succ n) = succ (succ (double n)) := by
  rfl
```

In the first two names, ”right” says that `zero` or `succ n` is the second argument of `add`. That is the argument examined by `match n with`, so these equalities follow directly from the definition. The analogous statements with `zero` or `succ` on the left need induction.

`unfold` makes a definition explicitly visible in the goal:

```
open Natural

example : double one = two := by
  unfold double
  unfold one two
  rfl
```

1.6 Rewriting with ‘rw‘

If we have `h : a = b`, `rw [h]` replaces `a` with `b` in the goal. `rw [← h]` uses the same equality in the opposite direction.

```
open Natural

theorem lecture06_rw_forward
    (a b : Natural)
    (h : a = b) :
    add a zero = b := by
  rw [h]
  rfl
```

After the rewrite, the left side is `add b zero`, which reduces to `b`. Conversely, if `h : a = b`, `rw [← h]` turns `add b zero = a` into `add a zero = a`.

```
open Natural

theorem lecture06_rw_backward
    (a b : Natural)
    (h : a = b) :
    add b zero = a := by
  rw ←[ h]
  rfl
```

1.7 Induction

`add m zero = m` reduces immediately because the second argument is explicitly `zero`. In `add zero n = n`, by contrast, `n` is a variable. Lean does not yet know whether `n` is `zero` or `succ k`, so it cannot choose a line of the definition of `add`.

Checking a few values is not enough because the result must hold for every natural number. A proof by induction has a *base case* for `zero` and an *inductive step*. We assume the result for `n` and prove it for `succ n`. The assumption in the step is the *induction hypothesis*.

In the step below, `rewrite [ih]` makes the same substitution as `rw [ih]`. Here it leaves the final goal visible, and we close it explicitly with `rfl`. `rw` can close a goal by itself when the rewritten goal has the form `t = t`.

```
open Natural

theorem lecture06_zero_add (n : Natural) :
    add zero n = n := by
  induction n with
  | zero =>
      rfl
  | succ n ih =>
      unfold add
      rewrite [ih]
```

```
      rfl
```

We choose the induction variable by looking at the recursive definition. Here `add` analyses its second argument. In the goal `add zero n = n`, we therefore induct on `n`.

The same choice applies to a goal with two arguments, such as `add (succ m) n = succ (add m n)`. The second argument of both additions is `n`, so we induct on `n`.

```
open Natural

theorem lecture06_succ_add (m n : Natural) :
    add (succ m) n = succ (add m n) := by
  induction n with
  | zero =>
      rfl
  | succ n ih =>
      unfold add
      rewrite [ih]
      rfl
```

`cases` and `induction` both split a value into its constructors. Only `induction` also provides an induction hypothesis in the successor case.

1.8 Associativity, commutativity, and ‘simp‘

For associativity, we choose `c`, the second argument of the outer addition:

```
open Natural

theorem lecture06_add_assoc (a b c : Natural) :
    add (add a b) c = add a (add b c) := by
  induction c with
  | zero =>
      rfl
  | succ c ih =>
      rw [lecture06_add_succ_right]
      rw [lecture06_add_succ_right]
      rw [lecture06_add_succ_right]
      rw [ih]
```

Commutativity does not follow from computation alone because `add` does not treat its two arguments symmetrically. In the base case we use `zero_add`. In the step, `succ_add` moves the successor from the first argument.

```
open Natural

theorem lecture06_add_comm (a b : Natural) :
    add a b = add b a := by
  induction b with
  | zero =>
```

```
      rw [lecture06_zero_add]
      rfl
  | succ b ih =>
      rw [lecture06_add_succ_right]
      rw [lecture06_succ_add]
      rw [ih]
```

`simp` automates rewrites that are oriented towards simpler forms. `simp only [lemmas]` explicitly limits the rules that may be used. `simp [lemma]` adds a rule for one command. `attribute [local simp] lemma` makes a rule available in the current section.

```
open Natural

theorem lecture06_zero_add_with_rw (n : Natural) : add zero n = n := by
  rw [lecture06_zero_add]
```

```
open Natural

theorem lecture06_zero_add_with_simp_only (n : Natural) : add zero n = n := by
  simp only [lecture06_zero_add]
```

```
open Natural

theorem lecture06_zero_add_with_simp (n : Natural) : add zero n = n := by
  simp [lecture06_zero_add]
```

```
open Natural

section LocalSimplification

attribute [local simp] lecture06_zero_add lecture06_succ_add

theorem lecture06_succ_add_with_simp (m n : Natural) :
    add (succ m) n = succ (add m n) := by
  simp

end LocalSimplification
```

```
open Natural

@[simp] theorem lecture06_double_zero :
    double zero = zero := by
  rfl
```

`@[simp]` registers a rule for the rest of the module. We use it for a canonical equation such as `double zero = zero`. Associativity and commutativity are not appropriate simp rules because they only move parentheses or swap arguments.

1.9 A final proof

We combine induction and rewriting to show that the double of a sum is the sum of the doubles. The subexpression that remains blocked is `add m n`, so we induct on `n`.

```
open Natural

theorem lecture06_double_add (m n : Natural) :
    double (add m n) = add (double m) (double n) := by
  induction n with
  | zero =>
      rfl
  | succ n ih =>
      rw [lecture06_add_succ_right]
      rw [lecture06_double_succ]
      rw [lecture06_double_succ]
      rw [lecture06_add_succ_right]
      rw [lecture06_add_succ_right]
      rw [ih]
```

After checking each step, we can express the same proof with a controlled collection of simplification rules:

```
open Natural

theorem lecture06_double_add_short (m n : Natural) :
    double (add m n) = add (double m) (double n) := by
  induction n with
  | zero =>
      rfl
  | succ n ih =>
      simp only [lecture06_add_succ_right, lecture06_double_succ, ih]
```

1.10 Working strategy

1. Identify the constructors of the type. 2. Read which argument is analysed by the definition. 3. Check whether the goal reduces with `rfl`. 4. Use previously proved equalities with `rw`. 5. If a variable remains in the analysed argument, choose that variable for induction. 6. Automate with `simp only` or `simp`.

1.11 Sources

- Lean developers, *Theorem Proving in Lean 4*, «Inductive Types»: https://lean-lang.org/theorem_proving_in_lean4/Inductive-Types/.
- Lean developers, *Theorem Proving in Lean 4*, «Induction and Recursion»: https://lean-lang.org/theorem_proving_in_lean4/Induction-and-Recursion/.
- Lean developers, *Functional Programming in Lean*, «Datatypes and Patterns»: https://lean-lang.org/functional_programming_in_lean/Getting-to-Know-Lean/Datatypes-and-Patterns/.

- Lean developers, *The Lean Language Reference*, «Recursive Definitions»: https://lean-lang.org/doc/reference/latest/Definitions/Recursive-Definitions/.
- Lean community, *Natural Number Game*: https://adam.math.hhu.de/.